

Put a web service under load

Session 3 – Web services, load & DDoS basics

Exercise 3 · Session 3 lab · Estimated time: 75–100 min

Write your answers in the answer document `SKY2100_Exercise3_Answers.docx`. Today you stand up a real web server, attack it with *controlled* load, and read what the metrics and logs show. You will watch a service degrade gracefully and then hit the wall – the same shape whether the load is real customers or a denial-of-service attack. This is your first hands-on encounter with the **Availability** leg of the CIA triad.

Caution: Run load *against your own VM only* (`localhost`). Pointing a load generator at a machine or site you do not own is an attack, and it is illegal.

1 Stand up a web server

▷ Install Nginx and the benchmark tool:

`sudo apt update && sudo apt install -y nginx apache2-utils`. Open `http://localhost/` in the VM browser and confirm the welcome page loads.

2 Generate and watch load

▷ Watch the system:

in terminal 1 run `htop` (or `bttop`); keep it visible.

▷ Generate load

(terminal 2): `ab -n 2000 -c 100 http://localhost/`. Record in the answer document: requests per second, time per request (mean), failed requests.

▷ Increase the pressure:

repeat with `ab -n 5000 -c 300 http://localhost/`. Note at roughly how many concurrent requests response time climbs sharply.

▷ Read the logs:

`sudo tail -n 20 /var/log/nginx/access.log`; correlate the burst of log lines with the spike you saw in `htop`.

Key idea: Watch what happened as you raised the concurrency. Requests per second climbed for a while, then flattened, and past a certain point response times shot up and requests began to fail. That is your VM meeting its capacity ceiling, and the shape of the curve is the same whether the load is real customers on a launch day or an attacker sending a flood. This is why availability is a security property, not just an operations concern: a denial-of-service attack does nothing clever, it simply pushes you past the point you just found. Note the concurrency where your own server broke, because every defence later in the course, from rate limiting to autoscaling, is about moving that point or soaking up load before it arrives.

▷ Interpret your run in 2–3 sentences: what happened to CPU/RAM and requests-per-second as concurrency rose?

Optional stretch – a first control

Add a rate limit (`limit_req_zone / limit_req`), reload Nginx, re-run the high-concurrency test, and compare the failed-request count before/after.

3 Cloud transfer: Microsoft Learn

-
- ▷ Complete “Describe Azure compute services” and “Describe Azure networking services”

(two modules in Part 2; the old combined module was split in 2026, and the compute module still lives at the old combined address).

learn.microsoft.com/training/modules/describe-azure-compute-networking-services

learn.microsoft.com/training/modules/describe-azure-networking-services

- ▷ Map today’s ideas to Azure services

(one line each in the answer document): Load Balancer (L4), Application Gateway (L7 + WAF), Traffic Manager / Front Door (global), Azure DDoS Protection.

4 Azure reflection

- ▷ Your local **ab** test hit one VM’s ceiling. Which Azure service would let the *same* application survive the same load, and what does it trade away (cost, complexity)?

- ▷ A **composite SLA**: if a solution chains a web app (99.95%) behind a gateway (99.95%), is the combined availability higher or lower than 99.95%? Explain in one line.

5 Knowledge check

1. A DDoS attack primarily targets which CIA property?

☐ Confidentiality ☐ Integrity ☐ Availability

Hint: It floods the service offline — which of the three properties is that?

2. Name the three flavours of DDoS and give one example of each.

Hint: By raw volume, by protocol tricks, and at the application layer.

3. **ab -n 1000 -c 50** means...

☐ 1000 concurrent requests ☐ 1000 requests total, 50 at a time ☐ 50 requests total
☐ 1000 servers

Hint: A thousand requests in total, fifty running at a time.

4. Why is an **application-layer** DDoS the hardest to tell apart from legitimate traffic?

Hint: Its requests are valid and, one by one, indistinguishable from real users.

5. An Azure **Application Gateway** operates at which layer, and which security control does it host?

Hint: Layer 7 — and it is where the WAF lives.

6. What is a **composite SLA**, and why can adding services *lower* overall availability?

Hint: Chained services multiply their availabilities, so the combined figure drops.

7. List *two* controls on your own server and *two* that work only at cloud scale, against DDoS.

Hint: Local: rate limits, caching, timeouts. Cloud: autoscaling, CDN, DDoS protection.

8. Define **capacity** and **availability**, and state the relationship between them.

Hint: One is work per second the system can do; one is being reachable when needed.

9. According to Comer (Ch. 1), what early problem led to clusters of servers behind a load balancer?

Hint: Early sites outgrew a single machine, so many ran behind a balancer.

10. Distinguish **scale-up** from **scale-out**, and say which the cloud favours.

Hint: A bigger single box versus more boxes — the cloud prefers more.

11. Multi-tenancy + autoscaling produces a “moving attack surface”. Explain in one sentence why that complicates security.

Hint: Instances come and go and tenants share hardware — there is no fixed list to guard.

12. (Reflection – from your data) At what concurrency did response time climb sharply, and what does that tell you about *your* VM's capacity?

Progress check: Discuss your answers in the group, then talk the teacher through them verbally at the next session only to track progress; nothing is handed in, signed or graded.

Solutions

Work through the lab first, then check here. Model answers are indicative — equivalent reasoning is fine.

Note: Model answers are indicative – accept equivalent reasoning. Multiple-choice answers are in **bold**.

Knowledge check

1. **Availability**.
2. **Volumetric** (UDP flood / amplification), **Protocol** (SYN flood), **Application-layer** (HTTP flood of an expensive endpoint).
3. **1000 requests total, 50 at a time**.
4. It uses small, valid-looking requests, individually indistinguishable from real users; only aggregate behaviour and logs reveal it.
5. **Layer 7** (application layer); it hosts the **WAF**.
6. A **composite SLA** is the combined SLA of chained services; multiplying availabilities (e.g. 0.9995×0.9995) yields a *lower* number than any component – so more components means lower combined availability.
7. Local: rate limiting, caching, timeouts and connection limits. Cloud-scale: autoscaling, CDN/anycast, cloud DDoS protection, WAF.
8. **Capacity** = work per unit time a system can do; **Availability** = reachable/ responsive when needed; availability fails once demand exceeds capacity.
9. Popular early web sites exceeded a single machine's capacity, so operators ran a *cluster* of identical servers behind a load balancer. (Comer Ch. 1.5, p. 8, verified.)
10. **Scale-up** = a bigger single machine; **scale-out** = more machines. The cloud favours **scale-out**.
11. Instances appear/disappear and tenants share hardware, so you cannot secure a fixed list of servers – only a pattern/policy; the surface changes continuously.
12. Reflection from each group's own measurement – no fixed value.

References

Comer Ch. 1 (clusters & load balancing **p. 8**, economic motivation **p. 10**) & Ch. 2 (multi-tenancy **p. 15**, elastic computing **p. 16**, IaaS **p. 19**, SaaS **p. 20**) – all verified; Jøsang Ch. 2 & 6 (syllabus, page numbers not verified). MS Learn: *Describe Azure compute services* and *Describe Azure networking services* (verified, June 2026 – the previously combined module was split into two).